

native platform layer. Compared to pure native apps, hybrid apps shine in terms of more platform support, ease and speed of development, and easy access to third party code.

As per its name, Ionic is like 'Bootstrap for Native'. It is a UI framework that handles the user interface and its interaction. Ionic comes with native style UI elements and layout.

A big advantage of using Ionic is that the code base remains one for all mobile platforms and the UI/UX looks like a native mobile platform. Ionic also uses AngularJS. It provides many directives for the Angular App.



Note: While developing mobile applications, do ensure that you have the Android SDK downloaded and configured. Similarly, at least one Android virtual device should also be configured. The same applies to iOS. Besides this, you also need a working Internet connection.

Installing and configuring the Ionic framework

All the examples and installations are done on an Ubuntu machine, though we can do the same on Windows or Mac machines, given that some commands for installation will differ. So all *apt-get* commands mean we are installing some packages or libraries. On Windows and Mac, the same can be downloaded and installed. A terminal in Ubuntu is the same as the command prompt in Windows.

The first step is to install Node.js and npm.

On a Linux machine, open the terminal and execute the following command:

```
$ sudo apt-get install nodejs npm
```

Before we move ahead, let's add *npm auto complete* in our *bashrc* file. This step is not compulsory, though.

```
$ npm completion >> ~/.bashrc
$ source ~/.bashrc
```

Let's install Ionic by executing the following commands, one by one:

```
$ npm install cordova gulp
$ npm install ionic
```

In the current working directory, you can see the folder name *node_modules*, which will have all that has been downloaded.

The next step is to configure the path for the mobile SDKs. For Android, we need to configure *ANDROID_HOME* to point to the location of the Android SDK, and add *platform-tools* and the *tools* path of the SDK to the *PATH* environment variable. Use the *set* command to set both the paths. Once that's done, use the *echo* command to verify the same, as shown below:

```
$ ashish@ashishwork:~$ echo $ANDROID_HOME
```

```
/home/ashish/android/sdk
$ ashish@ashishwork:~$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin:/home/ashish/android/sdk/platform-tools:/home/ashish/android/sdk/tools
```

Building a demo application using Ionic

To start the application, Ionic provides three basic blank templates, tabs and a side menu. We will start with the blank template. Open a terminal and run the following command:

```
$ ionic start myContact blank
```

This will get the blank project from the Net on to the local machine and configure it. Also, Cordova will get configured for the new application.

When asked to create the *ionic.io* account, type *n* for no, to skip the account creation as shown in Figure 1. This will create a folder named *myContact* in the current

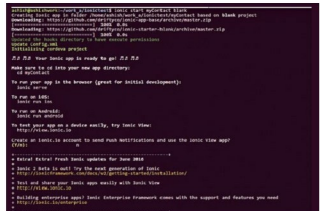


Figure 1: Starting a new blank project



Figure 2: Output of *myContact* app in browser

directory; move to the *myContact* using *cd myContact*. As mentioned in the introduction, since Ionic uses HTML5, we can test the output in the browser directly. In the terminal window, since we are already in the *myContact* path, type the command *ionic serve* and press *Enter*, which will open the browser with the *myContact* app as shown in Figure 2.

Coming back to the terminal, you will see the prompt changes to *ionic \$*, as shown in Figure 3.

Change the files and refresh the browser to see the changes. There's no need to compile or build the application. Once we are done with changes, press *q* on the terminal to return to a normal prompt. For now, press *q* in the terminal window.

Before moving ahead, let's build the application and test